Attracteur de Rössler

Sandry Léomima Randriamalala
Félix Oriel

avril 2025

Table des matières

1 Introduction **1**

2 Présentation des différents paramètres du programme **2**

3 Observation des cycles et des dédoublements de période **2**

4 Section de Poincaré **6**

5 Application de Poincaré et comportement chaotique du système de Rössler **6**

6 Constante de Feigenbaum (Essai) **8**

7 Conclusion **8**

Annexe : Code Python pour la section de Poincaré **8**

Table des figures

1 Trajectoire du système de Rössler pour $c = 1.5$, $a = 0.25$, $b = 1$. 3
2 Trajectoire du système de Rössler pour $c = 2.24$, $a = 0.25$, $b = 1$. 3
3 Premier dédoublement de période pour $c = 3.85$, avec $a = 0.25$, $b = 1$. 4
4 Deuxième dédoublement de période pour $c = 4.61$, avec $a = 0.25$, $b = 1$. 5
5 comportement semble chaotique 5
6 exemple section de Poincaré 6
7 fonction f_y 7

1 Introduction

L'objectif de ce travail est l'étude d'un système dynamique non linéaire : l'attracteur de Rössler. Ce système présente une transition intéressante entre comportement déterministe et comportement chaotique selon les valeurs des paramètres.

Il est défini par le système d'équations différentielles suivant :

$$\begin{cases} \dot{x} = -y - z \\ \dot{y} = x + ay \\ \dot{z} = b + (x - c)z \end{cases}$$

où a, b, c sont des paramètres réels.

L'attracteur de Rössler évolue dans un espace à trois dimensions $(x(t), y(t), z(t))$, et permet d'observer différents régimes dynamiques : trajectoires périodiques, bifurcations, comportements quasi-périodiques ou chaotiques.

Pour analyser ce système, nous utiliserons un programme Python fourni (`roessler.py`). Ce programme permet de visualiser l'évolution des trajectoires en fonction du temps et des paramètres.

Le travail se décompose en plusieurs étapes :

— **Observation des cycles et des doublements de période** : en faisant varier le paramètre c, on observe l'apparition successive de bifurcations de période menant au chaos.
— **Application de Poincaré** : nous étudions la dynamique réduite sur une section de l'espace des phases (plan $x = 0$) et analysons l'application discrète associée, permettant de mettre en évidence les propriétés du système.
— **Constante de Feigenbaum** : en mesurant les valeurs successives du paramètre c où apparaissent les doublements de période, nous estimons la constante de Feigenbaum, caractéristique universelle des transitions vers le chaos.

Ce travail nous permettra ainsi d'approfondir notre compréhension des systèmes dynamiques chaotiques, et d'illustrer expérimentalement certains concepts fondamentaux de la théorie du chaos.

2 Présentation des différents paramètres du programme

La simulation numérique du système de Rössler est pilotée par plusieurs paramètres définis dans le programme Python fourni. Leur bonne compréhension est essentielle pour interpréter correctement les résultats obtenus.

— **Paramètres dynamiques** (a, b, c) **:** Ce sont les paramètres internes du système de Rössler. Dans ce TP, les valeurs de $a = 0{,}25$ et $b = 1$ sont généralement fixées, tandis que c est le paramètre libre que l'on fait varier pour observer l'évolution du comportement dynamique (passage du régime périodique au chaos).
— **Condition initiale** (x_0, y_0, z_0) **:** Représentée dans le code par la variable `R_in`, elle définit la position de départ dans l'espace des phases. Une mauvaise condition initiale peut conduire à de longues transitions avant que le système ne se stabilise sur l'attracteur.
— **Temps initial `t0` :** Il s'agit d'un temps de "pré-intégration" (retard) avant l'affichage de la trajectoire. L'augmentation de ce temps permet de s'assurer que le point initial est bien situé sur l'attracteur et d'éliminer les tracés inutiles. Cela améliore la qualité des observations.
— **Durée d'intégration `t1` :** C'est le temps pendant lequel la trajectoire est tracée après le temps t_0. Une durée plus longue permet de mieux observer le comportement du système sur la section étudiée.
— **Nombre de points `N` :** Ce paramètre correspond au nombre de points calculés sur la trajectoire. Plus la valeur de `N` est grande, plus l'intégration est précise et la courbe lissée. Cependant, cela augmente aussi le temps de calcul et peut rendre l'interface plus lente si la valeur est trop élevée.
— **Paramètre `c` (modulable) :** C'est le paramètre principal que l'on fait varier pour observer les transitions du système. En modifiant c, on peut observer des cycles limites, des bifurcations de période, puis l'apparition du chaos. Le programme permet de faire varier ce paramètre dynamiquement à l'aide d'un curseur interactif.

Ces différents paramètres sont accessibles via une interface interactive intégrée au programme, permettant une exploration visuelle intuitive du comportement du système dynamique selon les réglages choisis.

3 Observation des cycles et des dédoublements de période

Dans cette partie, nous étudions l'évolution qualitative des trajectoires du système de Rössler en fonction du paramètre c. En augmentant progressivement la valeur de c, nous observons l'apparition de cycles limites stables, suivie de dédoublements de période successifs menant progressivement à un comportement chaotique.

Dans cette séction, on utilise les paramètre $a = 0.25$ et $b = 1$.

Comportement pour $c < 2$

Lorsque le paramètre c est inférieur à 2, le système de Rössler présente un comportement périodique stable. La trajectoire converge rapidement vers un cycle limite : une orbite fermée dans l'espace des phases, qui se répète indéfiniment. En 3D, la trajectoire forme une boucle fermée unique (ressemble à une spirale plate , à 1 seule nappe).

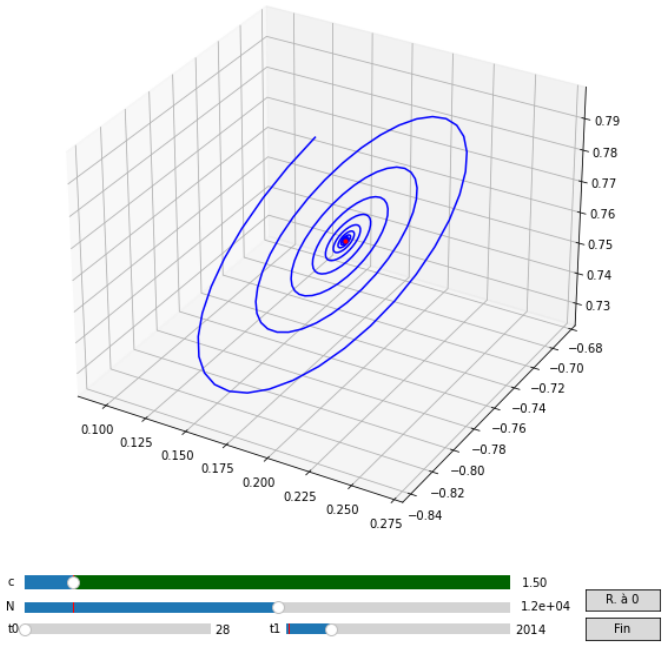

FIGURE 1 – Trajectoire du système de Rössler pour $c = 1.5$, $a = 0.25$, $b = 1$.

Le système converge vers un cycle limite stable. La trajectoire reste régulière et forme une boucle fermée dans l'espace des phases.
Ce comportement indique l'absence de chaos ou de bifurcation. Le mouvement est déterministe, prévisible.

Comportement pour $2 < c < 2.5$

Lorsque le paramètre c augmente au-delà de 2, tout en restant inférieur à 2.5, la trajectoire du système reste encore dans un régime périodique stable : il n'y a pas encore de dédoublement de période. Le système converge rapidement vers un cycle limite unique, mais celui-ci change progressivement de forme.

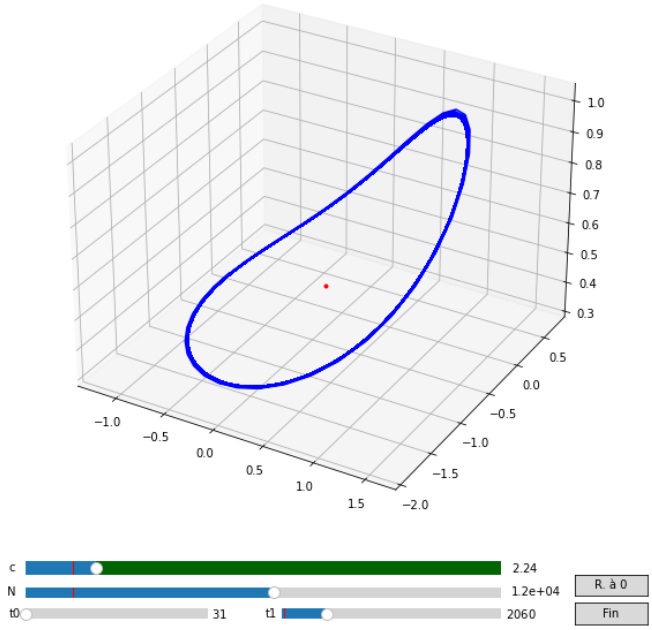

FIGURE 2 – Trajectoire du système de Rössler pour $c = 2.24$, $a = 0.25$, $b = 1$.

Comparé au cas $c < 2$, où la trajectoire forme une spirale presque plate, le cycle ici devient plus courbé, occupant un volume plus important dans l'espace des phases. Il semble commencer à se torsader, ce qui annonce une complexification progressive de la dynamique du système, bien que le comportement reste encore parfaitement déterministe.

Dédoublement de période : transition vers le chaos

On va partir maintenant de $c_1 = 2.5$ qui est cycle limite stable.
Lorsque le paramètre c dépasse une certaine valeur critique c_2, le système de Rössler subit une bifurcation : le cycle limite stable se dédouble. La trajectoire ne revient plus à son point initial à chaque tour, mais alterne entre deux branches (ou "nappes") distinctes — c'est le phénomène de dédoublement de période.

Ce phénomène marque le début d'un scénario de bifurcations successives, où la période du cycle est doublée à chaque étape : 1 période $\rightarrow$ 2 périodes $\rightarrow$ 4 périodes, etc. Ce processus, bien connu en dynamique non linéaire, conduit progressivement au chaos. On parle de cascade de bifurcations de période.

Dans cette section, nous cherchons à :

— Identifier expérimentalement la valeur c_2 à partir de laquelle le cycle se dédouble ;

— Observer ce nouveau régime et confirmer la périodicité double à l'aide des trajectoires ;

— Relever les valeurs suivantes $c_3, c_4, \ldots$ correspondant à d'autres dédoublements, si l'image le permet.

Nous utilisons ici toujours les paramètres $a = 0.25$, $b = 1$, et faisons varier c à partir de 2.5 jusqu'à observer les premiers signes de complexification.

Premier dédoublement de période ($c_2 = 3.85$)

Lorsque le paramètre c est augmenté jusqu'à environ $c_2 = 3.85$, on observe un phénomène de dédoublement de période : la trajectoire ne décrit plus un seul cycle fermé, mais alterne entre deux cycles distincts. Cela se traduit visuellement par deux boucles dans l'espace des phases, parcourues alternativement. Ce phénomène marque la première étape vers une dynamique chaotique.

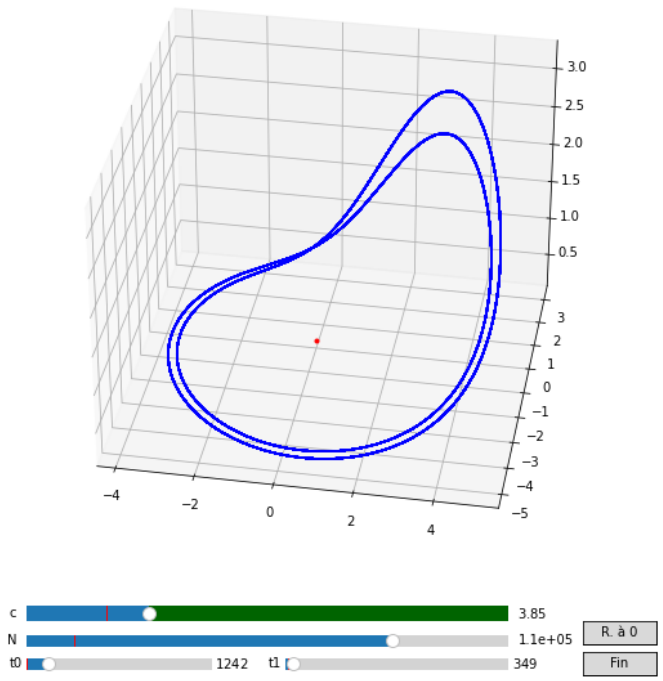

FIGURE 3 – Premier dédoublement de période pour $c = 3.85$, avec $a = 0.25$, $b = 1$.

Ce comportement correspond à une bifurcation de période 2, première étape d'une cascade de Feigenbaum menant au chaos.

Deuxième dédoublement de période ($c_4 = 4.61$)

Après le premier dédoublement de période observé pour $c_2 = 3.85$, nous observons un deuxième dédoublement de période lorsque le paramètre c atteint $c_4 = 4.61$. À ce stade, chaque cycle issu du premier dédoublement se dédouble à son tour, ce qui entraîne une périodicité plus complexe.

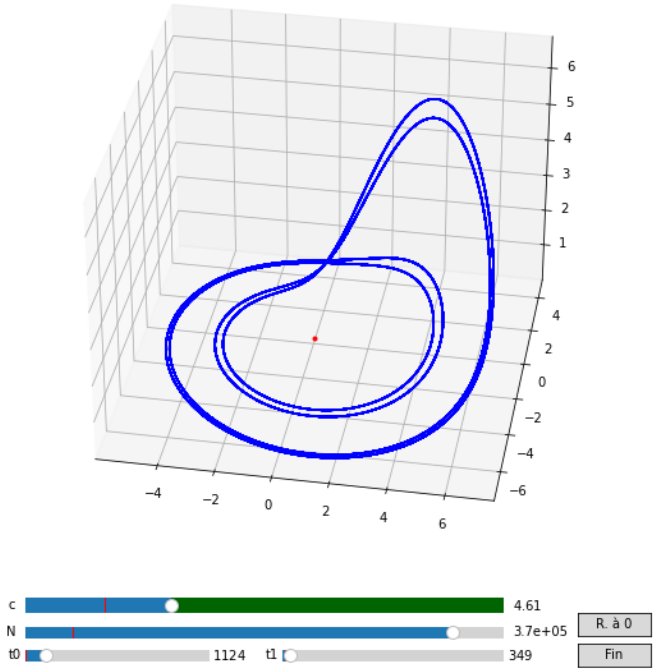

FIGURE 4 – Deuxième dédoublement de période pour $c = 4.61$, avec $a = 0.25$, $b = 1$.

Lorsque $c = 4.61$, les trajectoires ne suivent plus les deux cycles alternés du premier dédoublement, mais chaque cycle de ces deux nappes se subdivise en deux, créant ainsi une structure plus complexe.

Ce phénomène est un indicateur clé de l'apparition d'un comportement chaotique, où le système devient très sensible aux conditions initiales. Chaque dédoublement successif (période 1, 2, 4, 8, etc.) est une caractéristique fondamentale des systèmes dynamiques chaotiques, où la prévisibilité du système devient de plus en plus difficile.

La complexité du système augmente, et les trajectoires deviennent de plus en plus chaotiques.

Comportement Chaotique

Le système de Rössler évolue d'un comportement périodique à un comportement chaotique en fonction de la variation du paramètre c. Après avoir observé les différents dédoublements de période, on peut identifier une valeur seuil à partir de laquelle le comportement du système devient chaotique.

Seuil du comportement chaotique À partir de $c = 4.90$, les trajectoires deviennent chaotiques : elles se confondent et ne suivent plus de motifs réguliers distincts. Ce comportement est caractérisé par une sensibilité extrême aux conditions initiales, rendant les trajectoires imprévisibles.

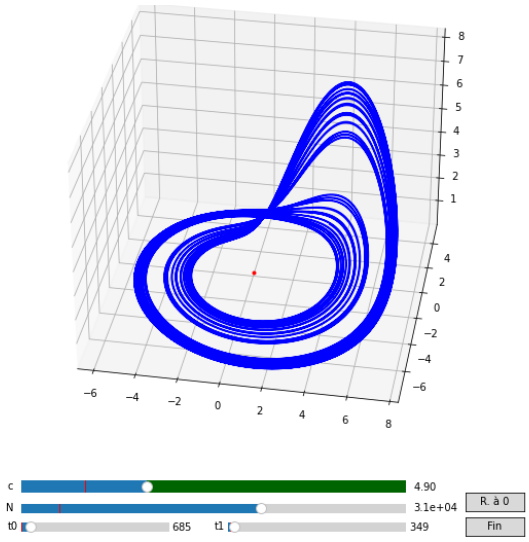

FIGURE 5 – comportement semble chaotique

Ainsi, la valeur seuil à partir de laquelle le comportement devient chaotique est $c \approx 4.90$. À ce niveau, le

système présente une dynamique complexe et non linéaire, typique des systèmes chaotiques.

4 Section de Poincaré

Dans l'étude des systèmes dynamiques comme celui de Rössler, la trajectoire évolue dans un espace continu à trois dimensions $(x(t), y(t), z(t))$, ce qui rend parfois difficile l'analyse précise de son comportement à long terme.

La **section de Poincaré** est une méthode puissante qui permet de réduire la dimension du système pour mieux comprendre sa dynamique. Elle consiste à observer les points d'intersection de la trajectoire avec un plan fixe, ici le plan $x = 0$, lorsque la trajectoire le traverse dans un certain sens .

Plutôt que d'étudier toute la trajectoire dans le temps, on ne retient que les points d'impact successifs sur ce plan. Cela transforme le système continu en une dynamique discrète, bien plus facile à interpréter.

Le code qui a été utilisé pour la faire est détaillé dans l'annexe.

Voici un exemple pour les paramètres suivants : $a = 0.25$,$b = 1$, $c = 5.5$.

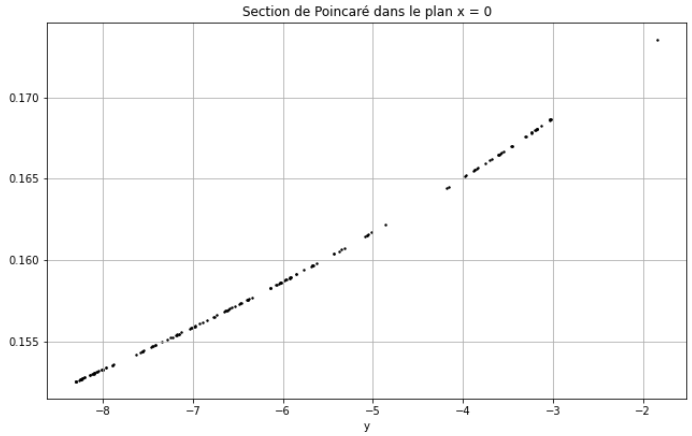

FIGURE 6 – exemple section de Poincaré

On va alors observer et étudier que les trous représentés dans la figure ci-dessus qui sont crées pendant le passage de la courbe sur le plan (ici $x = 0$).

5 Application de Poincaré et comportement chaotique du système de Rössler

L'étude des systèmes dynamiques tridimensionnels comme celui de Rössler peut être simplifiée par l'utilisation de la section de Poincaré. Celle-ci consiste à observer les points où une trajectoire coupe une surface de l'espace des phases, ici le plan $x = 0$, avec la condition que la trajectoire le traverse dans le sens croissant ($\dot{x} > 0$).

À chaque intersection de la trajectoire avec le plan $x = 0$, on enregistre la valeur de la variable y. Cela permet de construire une application discrète :

$$f : y_k \mapsto y_{k+1},$$

où y_k est la valeur de y au k-ième passage. Cette fonction f, représente la loi d'évolution de y sur la section de Poincaré.

La construction numérique de l'application de Poincaré suit les étapes suivantes :

1. Intégration du système de Rössler sur une longue durée à partir d'une condition initiale.
2. Détection des passages dans le plan $x = 0$ avec $\dot{x} > 0$.
3. Extraction des composantes y_k pour chaque point d'intersection.
4. Construction du nuage de points (y_k, y_{k+1}) correspondant à l'application f.

Le nuage de points obtenu donne une représentation graphique de l'application f.Les observations permettent de conclure au caractère chaotique du système :

— Dispersion du nuage de points indiquant une grande sensibilité aux conditions initiales.(méthode graphique).

— Analyse de la stabilité du point fixe : la pente $f'(y^*)$ au point fixe est estimée (méthode numérique). Si $|f'(y^*)| > 1$, le point fixe est instable, ce qui est un fort indicateur de chaos.

Avec le code qui est donné en annexe , on a pu tracé la fonction f_y.

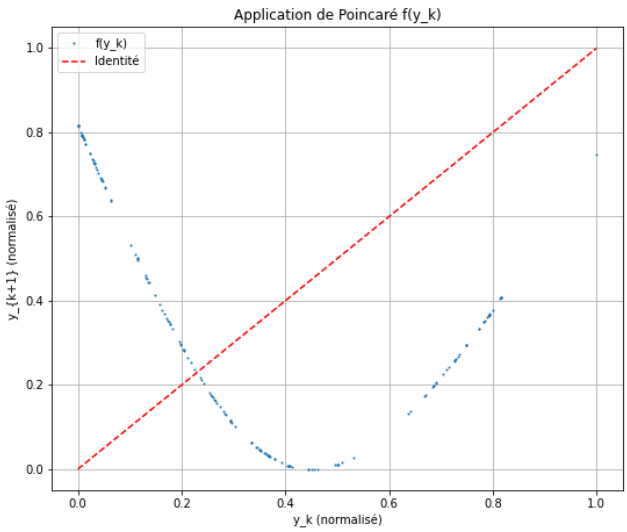

FIGURE 7 – fonction f_y

Pour analyser la stabilité du point fixe y^* de l'application de Poincaré f, nous avons estimé numériquement la dérivée $f'(y^*)$. Cette estimation s'est faite à partir du nuage de points (y_k, y_{k+1}) qui est expliquée dans l'annexe (code) :

Nous avons d'abord détecté approximativement le point fixe en identifiant l'intersection entre la courbe $y_{k+1} = f(y_k)$ et la diagonale $y_{k+1} = y_k$. Puis Autour de ce point y^*, nous avons sélectionné une petite fenêtre de données (y_k, y_{k+1}) proches de cette intersection. Et enfin une régression linéaire a ensuite été effectuée localement sur ces données pour approximer la pente $f'(y^*)$.

Résultat obtenu. À l'issue de l'estimation numérique, nous avons obtenu une pente locale de l'application de Poincaré au point fixe :$|f'(y^*)| \approx 2.01$.

Cette valeur, significativement supérieure à 1, indique que le point fixe est instable, ce qui confirme la présence d'un comportement chaotique dans le système pour les paramètres $a = 0{,}25$, $b = 1$, et $c = 5{,}5$.

Étude de l'application de Poincaré dans le plan (xOz) :

Nous étudions maintenant l'application de Poincaré dans le plan (xOz) pour le système de Rössler. L'objectif est de construire la suite $\tilde{x}_{k+1} = f_x(\tilde{x}_k)$ en extrayant les intersections successives avec le plan $y = 0$, dans le sens croissant (i.e., $\dot{y} > 0$), et en analysant la dynamique résultante. Pour ce faire, on extrait les points d'intersection avec le plan $y = 0$, ce qui permet d'obtenir les coordonnées (x_k, z_k) correspondant à chaque passage. À partir de ces points, on construit la suite $\tilde{x}_k$, et on évalue la fonction f_x qui permet de déterminer $\tilde{x}_{k+1}$ en fonction de $\tilde{x}_k$.

L'étude consiste ensuite à tracer le graphe de $f_x(\tilde{x}_k)$ en fonction de $\tilde{x}_k$, comparé à la diagonale $\tilde{x}_{k+1} = \tilde{x}_k$. Ce graphe permet d'analyser la stabilité et la nature de la dynamique : lorsque $f_x(\tilde{x}_k)$ est proche de la diagonale, le comportement est proche d'un point fixe ou périodique ; lorsqu'il s'en éloigne de manière irrégulière, cela reflète une dynamique chaotique.

Un aspect important de cette étude est l'identification d'une valeur critique $\tilde{x}_c$. Cette valeur sépare deux types de comportements topologiques : pour $\tilde{x}_k < \tilde{x}_c$, les points (x_k, z_k) décrivent une région qui correspond à un disque, représentant une dynamique relativement régulière ; en revanche, pour $\tilde{x}_k > \tilde{x}_c$, les points appartiennent à une région topologiquement différente, correspondant à un ruban de Möbius, ce qui traduit une complexité accrue et une perte d'orientation dans la dynamique.

En pratique, cette analyse nous permet de mieux comprendre les changements qualitatifs dans le comportement du système de Rössler, notamment la transition d'un régime stable à un régime chaotique, souvent visualisée par une bifurcation dans le graphe de l'application de Poincaré. Il se trouve qu'en raison de problèmes techniques lors du traitement des données (notamment une difficulté dans l'extraction fiable des points de section dans le

plan $y = 0$), On n'a pas pu mener l'expérience jusqu'à son terme (En fait la figure nous à montrer une valeur de x *critique* qui est fausse,$x = 0.66$). Cependant, la méthodologie établie reste valable pour toute analyse future visant à capturer cette transition entre disque et ruban de Möbius.

6 Constante de Feigenbaum (Essai)

Pour aller plus loin dans l'étude de ce système, on pourrait s'intéresser à la constante de Feigenbaum. C'est un nombre sans dimension qui est défini comme le rapport de distance entre deux biffurcations successives. Dans notre cas (ici, la distance est caractérisée par c) le nombre de Feigenbaum s'écrirait $\frac{C_{4n}-C_{2n}}{C_{2n}-C_n}$.

Plus le nombre de cycles est élevé, moins la distance entre deux bifurcations est importante. Ainsi, on pourrait observer que ce nombre, à l'approche du chaos, va tendre vers une constante qui d'après de nombreuses études, est la même pour la plupart des systèmes linéaires.

Paradoxalement, cette constante pourrait permettre de déterminer le passage d'un comportement déterministe au chaos. De plus, elle montre que le chaos nait de l'ordre, et que son passage semble universel au sein de différents systèmes pourtant bien différents.

Malheureusement, nous n'avons pas réussi à réellement étudier cette constante au sein de notre système. En effet, il est difficile de clairement identifier les doublements de période (cela assez rapidement) sur le graphe. Une solution envisageable serait de modifier le programme Python, afin d'avoir numériquement accès aux valeurs de c traduisant une bifurcation. En y parvenant, on pourrait avoir accès aux différents nombres de Feigenbaum et observé leurs comportements.

7 Conclusion

L'étude de l'attracteur de Rössler a permis de relater et de découvrir la transition d'un comportement déterministe à un comportement chaotique dans un système dynamique non linéaire.
Tout d'abord, en modifiant le paramètre c on a pu observé des changements de cycles conduisant à des résultats chaotiques. Ensuite, l'utilisation de l'application et de la section de Poincaré nous ont facilité l'étude de ce système. Nous permettant entre autre de mettre en évidence l'instabilité du point fixe. Cependant, l'étude de la constante de Feigenbaum n'a pas pu être correctement réalisé mais permet tout de même d'ouvrir de futurs pistes sur la compréhension des systèmes dynamiques.
Ce travail à finalement permis de nous rendre compte de la complexité d'un système qui pourtant parait simple, et nous montre à quel point les outils informatiques sont indispensables dans l'étude des comportements chaotiques.

Annexe : Code Python pour la section de Poincaré

Listing 1 – Code de la section de Poincaré

```python
def section_de_Poincare(r0, parametres, tmax=500, npoints=10000):
    """
    Calcule les points de la section de Poincare dans le plan x=0
    lorsque dx/dt > 0 pour le systeme de Rossler.

    Arguments :
        r0 : tuple representant la condition initiale (x0, y0, z0)
        parametres : tuple (a, b, c) du systeme de Rossler
        tmax : duree totale de la simulation (temps final)
        npoints : nombre total de points a integrer

    Retour :
        Liste des points (y, z) correspondant aux passages par x=0 avec dx/dt > 0
    """
    # Integration du systeme de Rossler avec les parametres donnes
    t, R = solve_Roessler(r0, parametres, tmax, npoints)

    # Separation des coordonnees x(t), y(t), z(t)
    X, Y, Z = R.T  # .T permet d'obtenir les vecteurs (x, y, z)

```

```python
    poincare_points = []  # Liste pour stocker les points de la section de Poincare

    # Parcours de tous les points (sauf le premier)
    for i in range(1, len(X)):
        # On detecte un passage par x = 0 dans le sens croissant (dx/dt > 0)
        if X[i-1] < 0 and X[i] > 0:
            # Interpolation lineaire pour estimer le point exact de passage
            alpha = -X[i-1] / (X[i] - X[i-1])

            # Interpolation des valeurs de y et z au moment exact du passage par x=0
            y_cross = Y[i-1] + alpha * (Y[i] - Y[i-1])
            z_cross = Z[i-1] + alpha * (Z[i] - Z[i-1])

            # On ajoute le point (y, z) e la section de Poincare
            poincare_points.append((y_cross, z_cross))

    # Retour sous forme de tableau numpy pour faciliter l'affichage
    return np.array(poincare_points)


def trace_section_Poincare(points):
    """
    Affiche les points de la section de Poincare dans le plan yOz.

    Argument :
        points : tableau numpy de points (y, z) obtenus par section_de_Poincare
    """
    # Creation de la figure
    plt.figure(figsize=(6, 6))
    plt.title("Section de Poincare dans le plan x = 0")

    # etiquettes des axes
    plt.xlabel("y")
    plt.ylabel("z")
    plt.grid(True)

    # Affichage des points sous forme de petits points noirs
    plt.plot(points[:, 0], points[:, 1], 'k.', markersize=2)

    # Affichage al'Ecran
    plt.show()
#--------------
# TEST DE LA SECTION DE POINCARE

poincare_pts = section_de_Poincare(R_in, (a, b, c), tmax=1000, npoints=100000)
trace_section_Poincare(poincare_pts)
```

Annexe : Code Python pour l'application de Poincaré

Listing 2 – Code pour l'étude de l'application de Poincaré

```python
def section_de_Poincare(r0, parametres, tmax=500, npoints=10000):
    """
    Calcule les points de la section de Poincare dans le plan x=0
    lorsque dx/dt > 0 pour le systeme de Rossler.

    Arguments :
        r0 : tuple representant la condition initiale (x0, y0, z0)
        parametres : tuple (a, b, c) du systeme de Rossler
        tmax : duree totale de la simulation
        npoints : nombre de points pour l integration
```

```python

    Retour :
        Liste des points (y, z) correspondant aux passages par x=0 avec dx/dt > 0
    """
    t, R = solve_Roessler(r0, parametres, tmax, npoints)
    X, Y, Z = R.T  # Transpose pour separer les composantes

    poincare_points = []
    for i in range(1, len(X)):
        if X[i-1] < 0 and X[i] > 0:
            alpha = -X[i-1] / (X[i] - X[i-1])  # interpolation lineaire pour x = 0
            y_cross = Y[i-1] + alpha * (Y[i] - Y[i-1])
            z_cross = Z[i-1] + alpha * (Z[i] - Z[i-1])
            poincare_points.append((y_cross, z_cross))
    return np.array(poincare_points)


def trace_section_Poincare(points):
    """
    Affiche la section de Poincare dans le plan y-z.

    Argument :
        points : tableau numpy de points (y, z)
    """
    plt.figure(figsize=(6, 6))
    plt.title("Section de Poincare dans le plan x=0")
    plt.xlabel("y")
    plt.ylabel("z")
    plt.grid(True)
    plt.plot(points[:, 0], points[:, 1], 'k.', markersize=2)
    plt.show()


def application_de_poincare_y(points):
    """
    Construit l'application de Poincare f : y_k -> y_{k+1}
       partir des points de la section.

    Argument :
        points : tableau numpy contenant les coordonnees (y, z)

    Retour :
        yk : valeurs de y      l instant   k
        yk1 : valeurs de y      l instant   k+1
    """
    y_vals = points[:, 0]
    yk = y_vals[:-1]
    yk1 = y_vals[1:]
    return yk, yk1


def normaliser(yk, yk1):
    """
    Normalise les valeurs de yk et yk1 entre 0 et 1
    pour faciliter   l analyse   graphique

    Arguments :
        yk : valeurs de y_k
        yk1 : valeurs de y_{k+1}

    Retour :
        yk_norm, yk1_norm : versions normalis es
    """
```

```python
    y_min = np.min(yk)
    y_max = np.max(yk)
    yk_norm = (yk - y_min) / (y_max - y_min)
    yk1_norm = (yk1 - y_min) / (y_max - y_min)
    return yk_norm, yk1_norm


def tracer_application_poincare(yk, yk1):
    """
    Trace le graphe de l'application f(y_k) = y_{k+1}
    et la droite identit .

    Arguments :
        yk, yk1 : points de l'application de Poincar
    """
    yk_norm, yk1_norm = normaliser(yk, yk1)
    plt.figure()
    plt.plot(yk_norm, yk1_norm, '.', markersize=2, label="f(y_k)")
    plt.plot([0, 1], [0, 1], 'r--', label="Identit ")  # f(y) = y
    plt.xlabel("y_k (normalis )")
    plt.ylabel("y_{k+1} (normalis )")
    plt.title("Application de Poincar  f(y_k)")
    plt.legend()
    plt.grid(True)
    plt.show()
    return yk_norm, yk1_norm


def calcul_pente_point_fixe(yk, yk1):
    """
    Calcule la pente de f au point fixe, i.e. l   o   f(y_k)     y_k.

    Arguments :
        yk, yk1 : donn es de l'application de Poincar

    Retour :
        y_fp : point fixe estim
        pente : valeur de la d riv e f'(y_fp)
    """
    yk_norm, yk1_norm = normaliser(yk, yk1)
    f_interp = interp1d(yk_norm, yk1_norm, kind='linear', fill_value="extrapolate")
    y_range = np.linspace(0, 1, 1000)

    # On cherche l'intersection entre f(y) et y (i.e. f(y) - y     0)
    diff = f_interp(y_range) - y_range
    idx = np.argmin(np.abs(diff))  # point de moindre  cart
    y_fp = y_range[idx]

    # Estimation de la d rivee autour de ce point
    pente = (f_interp(y_range[idx+1]) - f_interp(y_range[idx-1])) / (y_range[idx+1] -
        y_range[idx-1])
    print(f"\nPoint fixe normalise     {y_fp:.3f}, pente     {pente:.3f}\n")
    return y_fp, pente
```